Table des matières

Exercices corrigés 08 — Mettre en place l'intégration continue 1
Questions courtes 1
Questions pratiques 3
Diagnostics 3
Questions à développer 4
Mises en situation 4
Corrigé 5

Exercices corrigés 08 — Mettre en place l'intégration continue

Comment travailler ces exercices. Répondez seul, puis vérifiez avec le corrigé. Les exercices marqués [labo] portent sur ce que vous venez de faire au labo ; les autres vérifient que le cours est clair dans votre tête. Tout vient de la Théorie 08 et du Labo 08.

Les repères. Les commandes dbt et sqlfluff se lancent depuis `Projet_Boreal_intermediaire/02_ae`, les requêtes DuckDB depuis `Projet_Boreal_intermediaire`, sur l'entrepôt du poste, sauf quand l'énoncé nomme l'entrepôt jetable de la CI, `ci.duckdb`. Un essai qui casse le projet se défait par `git restore` avant la question suivante.

Le corrigé suit les énoncés, en fin de document. Répondez à l'énoncé en entier avant d'y passer : c'est en butant sur une question qu'on découvre ce qu'on avait lu sans le comprendre.

Questions courtes

1. La revue d'un modèle. Nommez cinq des questions qu'une revue pose à un modèle dbt, et dites pourquoi la syntaxe n'en fait pas partie.
2. [labo] Ce qu'un changement touche. Dites où l'on apprend ce qui dépend de `fct_ventes`, donnez le nombre de nœuds qui en dépendent directement, et nommez-en les familles.
3. L'intégration continue. Définissez-la en une phrase, et dites ce qu'elle protège.
4. CI, CD et orchestration. Distinguez les trois, et dites lequel des trois Boréal n'automatise pas, et pourquoi c'est défendable.
5. Les déclencheurs et les droits. Dites à quelle question répond chacun des deux déclencheurs du workflow, et ce que fait la ligne `contents: read`.

© 2026 iAlexMG. Tous droits réservés.

6. Le runner et les versions. Dites ce que « neuf à chaque exécution » impose au workflow, et pourquoi `ci/requirements.txt` épingle ses versions quand celui de la formation ne le fait pas.
7. [labo] L'ordre des étapes. Dites ce qui arrive si le linteur passe avant `dbt deps`, et pourquoi.
8. Le templater et son profil. Dites pourquoi sqlfluff compile le projet avant de le lire, où se déclare le profil qu'il lui faut, et ce que rend l'option `--profiles-dir`.
9. Les clés mal rangées. Décrivez les deux façons de mal ranger le profil du linteur, et le symptôme de chacune sur le poste et sur le runner.
10. [labo] Les 46 écarts. Nommez les cinq codes du premier passage du linteur, puis dites combien `sqlfluff fix` en corrige et combien il en laisse.
11. Les deux exclusions. Dites ce que veulent `ST06` et `ST09`, et pourquoi le projet les écarte.
12. `RF02`. Dites pourquoi une colonne non qualifiée dans une sous-requête n'est pas une affaire de style, avec le chiffre mesuré.
13. `-- noqa` et la Jinja. Dites quand on écrit `-- noqa: RF01` et pourquoi avec son code, puis dites ce que `template_blocks_indent = false` évite.
14. [labo] Le correcteur. Dites ce que `sqlfluff fix` a fait de `dim_date`, et la règle de conduite qui en découle.
15. La structure vide. Dites ce que contient `ci/structure_sources.sql`, comment on le fabrique, et pourquoi la CI n'a pas les données.
16. [labo] Ce que la CI bâtit. Donnez le nombre de nœuds, d'objets bâtis et d'objets qui ont des lignes, et nommez ces derniers.
17. Les tests de la CI. Dites pourquoi les 98 tests de données passent sur des tables vides, et nommez les deux sortes de tests qui voient quelque chose sans données.

1. La cible `ci`. Dites pourquoi son chemin est relatif, dans quels schémas ses modèles atterrissent, et pourquoi le snapshot n'y est pas exclu.
2. Le ménage. Dites les deux choix qui rendent `nettoyer_dev` sûre, et ce qu'elle ne touche pas.
3. Le profil versionné. Dites pourquoi versionner `ci/profiles.yml` ne contredit pas le Cours 03, et ce que le `.gitignore` du Cours 01 y opposait.
4. La fraîcheur. Dites ce que `dbt source freshness` compare, puis donnez les seuils de Boréal, source par source, avec leur raison.
5. Le seuil rond. Dites ce que des seuils de 36 et 72 heures sur toutes les sources ont donné sur la copie vieillie, et pourquoi c'est une panne d'alerte.
6. L'avertissement et la CI. Dites ce qu'un `WARN` seul fait au code de sortie et au verdict de la CI, et la règle qui en découle. Dites aussi pourquoi la fraîcheur ne va pas dans la CI, avec le chiffre.
7. [labo] Le contrat. Dites ce qu'il vérifie et quand, ce que devient `quantite::integer` avec et sans lui, et ce qu'il ne vérifie pas.
8. Le manifeste. Dites ce qu'un agent y lit, et la limite qu'il faut savoir énoncer.

© 2026 iAlexMG. Tous droits réservés.

Questions pratiques

P1. [labo] Le lignage en ligne de commande. Écrivez la commande dbt qui liste tout ce qu'un changement à `fct_ventes` touche, puis celle qui n'en garde que les modèles et l'`exposure`. Donnez les deux comptes, et dites pourquoi la première trouve un nœud de plus que les liens directs du manifeste.

P2. L'âge des dix sources. Écrivez la requête DuckDB qui donne, pour chacune des dix sources surveillées, l'âge de son dernier chargement en heures et le verdict que `dbt source freshness` rendra, sans lancer dbt. Donnez le résultat sur votre entrepôt, et dites ce que la requête ne sait pas faire que dbt fait.

P3. Un contrat sur le résumé. `rpt_ventes_par_palier` est le modèle que le tableau de bord lit. Posez-lui un contrat, en trouvant d'abord le type réel de ses sept colonnes. Dites quel piège de type vous attend sur les deux mesures, et ce que rend la CI si vous tombez dedans.

P4. (avancé) La CI mince, en local. Gardez le `manifest.json` de l'état actuel dans un dossier à part, rendez la borne de fin de `fct_ventes` inclusive, puis écrivez la commande qui liste les nœuds que la CI mince reconstruirait. Donnez le compte. Lancez ensuite la construction de cette sélection sur un entrepôt jetable neuf, et dites ce qui manque pour qu'elle marche.

P5. La CI en une commande. Écrivez `ci/verifier.cmd`, un script pour l'Anaconda Prompt qui rejoue les étapes de la CI dans l'ordre depuis la racine du dépôt, repart d'un entrepôt jetable neuf, et s'arrête au premier échec. Donnez sa durée et son comportement sur la branche de l'étape 12.

P6. Un renommage sous contrat. Renommez la colonne `cout` de `fct_ventes` en `cout_standard`, dans le modèle seulement. Donnez ce que rend la CI, puis décrivez la procédure qui mène ce renommage à bien sans casser personne.

P7. Ce que le manifeste ne dit pas. Écrivez le script Python qui compte, dans `target/manifest.json`, les modèles sans description, les colonnes déclarées sans description et les sources sans description. Donnez les comptes, et dites ce qu'un agent de documentation ferait de ce résultat.

P8. (avancé) Le seuil qui se mesure. Écrivez la requête qui donne, pour la télémétrie, le nombre de jours de réception, le plus long intervalle entre deux jours de réception et sa date. Donnez le résultat, puis dites si le seuil de 72 heures aurait sonné ce jour-là, et si cela change le réglage.

Diagnostics

E1. Au vert sur mon poste. Une collègue range `profiles_dir = ci` et `target = ci` sous `[sqlfluff]` plutôt que sous `[sqlfluff:templater:dbt]`. Sur son poste, `sqlfluff lint` passe. Elle pousse, et l'étape « La forme » tombe sur GitHub. Dites pourquoi les deux verdicts diffèrent, avec le message du runner, et dites ce que son poste a lu à la place du profil de la CI.

E2. La CI qui laisse passer dix-neuf millions. Un collègue retire, dans `int_lignes_enrichies`, le filtre qui écarte les postes refusés : « les ventes perdues sont des ventes, la direction veut les voir ». Sa demande de tirage passe au vert. Dites pourquoi, dites ce que le fait devient sur les vraies données, avec les chiffres, et dites quel test l'attrape et où il aurait fallu qu'il tourne.

© 2026 iAlexMG. Tous droits réservés.

E3. L'alerte qui hurle. Une équipe règle la fraîcheur de toutes ses sources à 36 et 72 heures, « comme dans l'exemple ». Au bout d'une semaine, lassée du rouge, elle passe toutes les erreurs en avertissements. Dites ce que chacun des deux réglages donne sur la copie vieillie de la théorie, avec les chiffres, et dites ce que l'équipe a perdu.

E4. La fraîcheur dans la CI. Un collègue ajoute au workflow une étape `dbt source freshness --target ci --profiles-dir ci`, pour que « la CI vérifie aussi que les données sont fraîches ». Dites ce que rend l'étape, à chaque exécution, et dites où cette commande doit tourner.

E5. Le contrat qui gênait. Un collègue renomme `cout` en `cout_standard` dans `fct_ventes`. La CI tombe sur le contrat. Il retire les trois lignes du contrat, et la CI repasse au vert. Dites ce que la CI rend alors, dites ce qu'il a cassé et pourquoi rien, dans le projet, ne le voit.

Questions à développer

A. Une CI sans données. Défendez ou critiquez le choix de construire la CI sur la structure vide des sources plutôt que sur un échantillon versionné. Dites ce que chaque choix vérifie, ce qu'il coûte, et ce que vous feriez pour que les défauts de données ne passent pas entre les mailles.

B. Avertir ou échouer. Un projet analytique a des tests, des seuils de fraîcheur et une CI. Expliquez comment vous décidez de ce qui avertit et de ce qui échoue, qui lit les avertissements, et ce qui arrive à un projet où plus personne ne les lit.

C. (avancé) Un agent qui relit les demandes de tirage. On vous propose un agent qui commente chaque demande de tirage du projet en s'appuyant sur le `manifest.json`. Dites ce qu'il peut attraper, ce qu'il ne peut pas, en vous appuyant sur les défauts de ce cours, et comment vous jugeriez qu'il vaut sa place.

Mises en situation

M1. Transport scolaire de Rimouski. Trois analystes font évoluer un projet dbt d'une cinquantaine de modèles. Tous poussent directement sur `main`, les tests existent mais personne ne les lance avant de pousser, et deux rapports ont été faux le mois dernier. On vous demande de « mettre de l'ordre ». Dites ce que vous montez en premier, ce que vous laissez pour plus tard, et la seule chose que vous refusez de sacrifier.

M2. (avancé) Fromagerie de Sherbrooke. Le projet dbt tourne sur Snowflake, et sa CI se connecte avec le mot de passe d'un utilisateur. Chaque demande de tirage reconstruit les quatre cents modèles du projet, et la facture de l'entrepôt de la CI a doublé en un an. Dites ce que vous changez à l'authentification, à ce que la CI reconstruit et à ce qu'elle lit, et ce que chaque changement exige.

M3. Meunerie de Montmagny. Un fournisseur dépose chaque lundi un fichier de prix des grains, parfois le mardi quand le lundi est férié. L'entrepôt le charge dans l'heure. On vous demande une alerte « si le fichier ne vient pas ». Dites quels seuils vous posez, sur quelle colonne, où la vérification tourne, et ce que vous écrivez à côté des chiffres.

© 2026 iAlexMG. Tous droits réservés.

Corrigé

Comment lire ce corrigé. Il donne la réponse et le pourquoi, jamais une notion neuve : tout ce qu'il emploie a été enseigné dans la Théorie 08 ou pratiqué dans le Labo 08. Les chiffres sortent d'exécutions réelles sur le projet de Boréal tel que le Labo 08 le laisse, sur son entrepôt jetable et sur la copie vieillie de la section 8 de la théorie.

Questions courtes

1. La revue d'un modèle. Le grain est-il déclaré et prouvé par un test ; `ref()` et `source()` sont-ils partout ; la logique vit-elle au bon étage ; le modèle a-t-il une description ; un test couvre-t-il le changement ; qu'est-ce qui dépend du modèle ; reste-t-il un secret ou un chemin personnel. La syntaxe n'en fait pas partie parce que la machine s'en charge : le linteur la vérifie à chaque demande de tirage, et une revue qui la relit passe à côté de ce qui casse vraiment un entrepôt, une borne, un grain, une règle au mauvais étage. *Théorie : `Theorie_08_AE.md`, section 1.*

2. Ce qu'un changement touche. Dans le graphe du projet, que le `manifest.json` écrit et que `dbt ls --select fct_ventes+` interroge. Douze nœuds dépendent directement de `fct_ventes` : le résumé `rpt_ventes_par_palier`, l'`exposure` du tableau de bord, neuf tests de données et le test unitaire. C'est la liste à avoir sous les yeux en relisant un changement à ce modèle. *Labo : `Labo_08_AE.md`, étape 14.*

3. L'intégration continue. Rejouer automatiquement les vérifications du projet à chaque changement proposé. Elle protège votre attention plutôt que votre code : elle change une discipline, lancer `dbt build` avant de fusionner, en garantie, qui tient la dixième fois comme la première, et quand un autre que vous propose le changement. *Théorie : `Theorie_08_AE.md`, section 2.*

4. CI, CD et orchestration. La CI vérifie le code à chaque changement proposé ; le déploiement continu pousse en production ce qui a passé la CI ; l'orchestration fait circuler la donnée à heure fixe, après le chargement. Boréal n'automatise pas le déploiement : la production se reconstruit par la tâche mensuelle, quand on le décide. C'est défendable sur un projet analytique, où l'on veut choisir le moment où les chiffres publiés changent, pas le laisser à l'approbation d'une revue. *Théorie : `Theorie_08_AE.md`, section 2.*

5. Les déclencheurs et les droits. La demande de tirage vers `main` demande si un changement peut entrer ; la poussée sur `main` demande si ce qui est entré tient toujours. `contents: read` réduit le jeton que GitHub prête à la tâche à la lecture du dépôt : une vérification n'a rien à écrire, donc elle ne reçoit pas le droit de le faire. *Théorie : `Theorie_08_AE.md`, section 3.*

6. Le runner et les versions. Le runner ne connaît ni l'environnement conda, ni dbt, ni le profil, ni l'entrepôt : tout s'installe ou se fabrique dans le fichier, d'où sa longueur. Il installe aussi la dernière version du jour de ce qu'on lui laisse choisir. `ci/requirements.txt` épingle donc, `dbt-duckdb==1.11.0`, pour que la CI ne change pas de verdict un matin sans que personne ait touché au code ; le `requirements.txt` de la formation garde ses fourchettes parce qu'il installe un poste, pas une preuve. *Théorie : `Theorie_08_AE.md`, section 3.*

7. L'ordre des étapes. Le linteur s'arrête avant d'avoir lu une ligne, sur `UninstalledPackagesFoundError` : son templater compile le projet, le projet appelle `dbt_utils`, et le paquet n'est installé que par `dbt deps`. Sur le poste, le dossier `dbt_packages/` masque le problème ; sur une copie neuve, il saute aux yeux. *Labo : `Labo_08_AE.md`, étape 10.*

© 2026 iAlexMG. Tous droits réservés.

8. Le templater et son profil. Un modèle dbt est de la Jinja qui produit du SQL : le linteur doit compiler pour lire ce qui sera exécuté. Compiler demande un profil et une cible, déclarés dans `.sqlfluff` sous `[sqlfluff:templater:dbt]`, par `profiles_dir` et `target`. L'option en ligne de commande n'existe pas : `sqlfluff lint --profiles-dir ci` rend `Error: No such option '--profiles-dir'.` et le code 2. *Théorie : `Theorie_08_AE.md`, section 4.*

9. Les clés mal rangées. `profiles_dir` seul sous `[sqlfluff]` : sur le poste, « The profile 'boreal_ae' does not have a target named 'ci' », parce que le linteur lit le profil personnel, qui n'a pas de cible `ci` ; sur le runner, « Could not find profile named 'boreal_ae' ». `profiles_dir` et `target` sous `[sqlfluff]` : sur le poste, rien, le linteur passe en compilant avec la cible `dev` du profil personnel ; sur le runner, le même « Could not find profile ». La seconde est la plus chère, parce qu'elle est verte là où l'on regarde. *Théorie : `Theorie_08_AE.md`, section 4.*

10. Les 46 écarts. `LT02`, l'indentation, 38 fois ; `RF02`, une colonne non qualifiée, 3 ; `RF01`, une référence introuvable, 2 ; `LT09`, plusieurs colonnes sur la ligne du `select`, 2 ; `LT14`, un `where` qui ne commence pas sa ligne, 1. `sqlfluff fix` en corrige 41 dans 12 fichiers et en laisse 5, les trois `RF02` et les deux `RF01`. *Labo : `Labo_08_AE.md`, étapes 2 et 3.*

11. Les deux exclusions. `ST06` veut les colonnes simples avant les calculs : elle range les colonnes par forme d'expression, et sa correction pousse les deux dates de `stg_commandes` après `lot_source`, contre l'ordre de conception du modèle. `ST09` veut la table nommée en premier à gauche d'une condition de jointure, et retournerait quinze jointures sans rien corriger. Aucune ne corrige un défaut, et leur raison est écrite à côté de la ligne qui les écarte. *Théorie : `Theorie_08_AE.md`, section 4.*

12. `RF02`. Une colonne sans préfixe dans une sous-requête se cherche dans la table de la sous-requête, puis dans la requête qui l'entoure, sans prévenir. `where cle_produit in (select cle_produit from marts.dim_date)` rend 80 618 lignes, tout le fait : `dim_date` n'a pas de `cle_produit`, la requête prend celle du fait et compare chaque ligne à elle-même. Un préfixe l'aurait fait échouer sur le champ. *Théorie : `Theorie_08_AE.md`, section 4.*

13. `-- noqa` et la Jinja. Sur un faux positif qu'on a regardé : `RF01` prend pour une table `etape`, la colonne de type structure que le latéral du Cours 06 déplie. Avec son code, le commentaire ne fait taire que cette règle-là sur cette ligne ; sans code, il les ferait toutes taire. `template_blocks_indent = false` empêche le correcteur de réindenter le contenu des blocs Jinja, dont il ne voit que la branche prise : sans lui, il trouve seize écarts dans le seul snapshot, en corrige quinze et le laisse à moitié décalé. *Théorie : `Theorie_08_AE.md`, section 4.*

14. Le correcteur. Il a rendu la liste des mois et celle des jours conformes et illisibles, l'indice coupé de son crochet sur deux lignes, `][month(date_day)` puis `] as nom_mois`. La règle : `fix` n'est pas une revue, et son `git diff` se relit en entier avant de commiter, comme le travail d'un collègue pressé. *Labo : `Labo_08_AE.md`, étape 3.*

15. La structure vide. La création des deux schémas `raw` et `argent` et de leurs 25 tables, en 27 lignes, sans une ligne de données. Une requête DuckDB la tire du catalogue de l'entrepôt, dont la colonne `sql` de `duckdb_tables()` porte l'instruction de chaque table. La CI n'a pas les données parce qu'elles appartiennent à Data Engineering, qu'elles ne doivent pas partir sur une machine publique, et qu'un échantillon demanderait quelqu'un pour le tenir. *Théorie : `Theorie_08_AE.md`, section 5.*

16. Ce que la CI bâtit. 130 nœuds, `PASS=129` et l'`exposure` en `NO-OP`. 30 objets : quinze tables dans `dev_marts`, treize vues dans `dev_staging`, une vue dans `dev_presentation` et le snapshot. Trois ont des lignes : le seed des anomalies et `dim_anomalie`, 5 chacun, et `dim_date`, 1 551 jours, qui ne dépendent pas de la `RAW`. *Labo : `Labo_08_AE.md`, étape 8.*

© 2026 iAlexMG. Tous droits réservés.

17. Les tests de la CI. Un test de données est une requête qui ne doit rien rendre, et une requête sur une table vide ne rend rien : les 98 passent sans rien vérifier. Le test unitaire voit quelque chose parce qu'il apporte ses propres lignes, et le contrat parce qu'il regarde des noms et des types, qui existent sur une table vide. *Théorie : `Theorie_08_AE.md`, section 5.*

18. La cible `ci`. Son chemin est relatif parce que la CI se lance toujours depuis la racine du dépôt, et qu'un chemin absolu désignerait un dossier du poste que le runner n'a pas. Ses modèles atterrissent dans `dev_staging`, `dev_marts` et `dev_presentation`, puisque la macro du Cours 07 préfixe toute cible qui n'est pas `prod`. Le snapshot n'y est pas exclu : sur un entrepôt neuf, il n'a pas d'histoire à abîmer, et `dim_concessionnaire` a besoin que sa table existe. *Théorie : `Theorie_08_AE.md`, section 6.*

19. Le ménage. Elle ne nomme aucun schéma en dur, elle les demande à `generate_schema_name`, et suit donc la convention quand elle change ; et elle commence par un verrou qui refuse la cible `prod`, code 1, avant de toucher à quoi que ce soit. Elle ne touche pas au schéma `snapshots`, partagé entre les cibles, dont l'histoire ne se reconstruit pas. *Théorie : `Theorie_08_AE.md`, section 6.*

20. Le profil versionné. Ce n'est pas le même fichier : le profil personnel porte un chemin du poste, et porterait un compte sur une plateforme ; celui de la CI ne porte qu'un nom de fichier relatif, et sur une plateforme que des `env_var(...)`. Le `.gitignore` du Cours 01 l'ignorait par sa ligne `profiles.yml`, qui vise tout fichier de ce nom, et ne disait rien de `ci.duckdb`, qui serait entré au premier `git add .`. *Théorie : `Theorie_08_AE.md`, section 7.*

21. La fraîcheur. La plus récente valeur de `_charge_le` de chaque source, contre l'heure qu'il est, et deux seuils, `warn_after` et `error_after`. L'ERP et les garanties, livrés une fois par mois : 35 et 40 jours, parce que l'âge normal va de zéro à trente et un. La télémétrie, livrée chaque nuit : 36 et 72 heures. Les actifs et les étiquettes de l'usine, exports complets livrés une fois : `freshness: null`, leur âge ne dit rien. `argent` n'a pas de colonne de chargement et se surveille chez Data Engineering. *Théorie : `Theorie_08_AE.md`, section 8.*

22. Le seuil rond. Dix `ERROR STALE` et le code 1, dont huit sur des sources parfaitement à jour : sur une source mensuelle, 72 heures la mettent en erreur du troisième jour à la fin du mois, vingt-sept jours sur trente. Une équipe qui voit rouge tous les jours cesse de regarder, et le jour où une source tombe vraiment, personne ne le voit. *Théorie : `Theorie_08_AE.md`, section 8.*

23. L'avertissement et la CI. Un `WARN` seul rend 0, et une étape qui rend 0 passe au vert : mesuré, 9 `PASS` et 1 `WARN` donnent le code 0. La règle : ce qui doit bloquer est une erreur, et un avertissement ne sert que si quelqu'un lit le journal. La fraîcheur ne va pas dans la CI parce que, sur l'entrepôt vide du runner, les dix sources sont périmées par construction, 10 `ERROR STALE` et le code 1 à chaque exécution ; elle mesure les données, et sa place est dans la tâche planifiée, avant `dbt build`. *Théorie : `Theorie_08_AE.md`, section 8.*

24. Le contrat. Les noms, les types et le nombre des colonnes que le modèle produit, contre ceux déclarés, avant de créer la table. Avec lui, `quantite::integer` fait tomber la construction, `PASS=117 ERROR=1 SKIP=12`, `INTEGER` contre `DECIMAL(18,3)` ; sans lui, tout passe et la colonne devient `INTEGER` en silence, aucune des 80 618 quantités n'ayant de décimale. Il ne vérifie aucun contenu : un fait aux bons types et aux mauvais montants le respecte. *Labo : `Labo_08_AE.md`, étapes 6 et 8.*

25. Le manifeste. Ce que le projet déclare : ses modèles, leurs descriptions, leurs colonnes et leurs types, ce que chacun lit et qui le lit, les tests qui le gardent. Un agent y trouve la structure de l'entrepôt sans l'ouvrir. La limite : il ne dit que ce que vous y avez écrit, et sur un projet sans descriptions ni tests, l'agent produit des réponses plausibles et fausses. *Théorie : `Theorie_08_AE.md`, section 10.*

© 2026 iAlexMG. Tous droits réservés.

Questions pratiques

P1. Le lignage en ligne de commande.

```
dbt ls --select fct_ventes+
dbt ls --select fct_ventes+ --resource-type model exposure
```

La première rend 14 lignes : `fct_ventes` lui-même et les 13 nœuds en aval. La seconde en rend 3 : `fct_ventes`, `rpt_ventes_par_palier` et l'`exposure` `tableau_de_bord_des_ventes`. Le manifeste compte douze liens directs ; `+` suit le graphe jusqu'au bout, et ajoute le treizième, le test `not_null` de `rpt_ventes_par_palier`, qui dépend du résumé et non du fait. Un changement à `fct_ventes` peut le faire tomber, et il faut le savoir en revue. *Théorie : `Theorie_08_AE.md`, section 1.*

P2. L'âge des dix sources.

```
with seuils (source, avertir_h, echouer_h) as (
    values
        ('erp_kna1', 35 * 24, 40 * 24),
        ('erp_mara', 35 * 24, 40 * 24),
        ('erp_vbak', 35 * 24, 40 * 24),
        ('erp_vbap', 35 * 24, 40 * 24),
        ('erp_t005u', 35 * 24, 40 * 24),
        ('erp_zserie', 35 * 24, 40 * 24),
        ('garanties_clients', 35 * 24, 40 * 24),
        ('garanties_enregistrements', 35 * 24, 40 * 24),
        ('garanties_reclamations', 35 * 24, 40 * 24),
        ('telemetrie_sorties', 36, 72)
),

ages as (
    select 'erp_kna1' as source, max(_charge_le) as dernier from raw.erp_kna1
    union all select 'erp_mara', max(_charge_le) from raw.erp_mara
    union all select 'erp_vbak', max(_charge_le) from raw.erp_vbak
    union all select 'erp_vbap', max(_charge_le) from raw.erp_vbap
    union all select 'erp_t005u', max(_charge_le) from raw.erp_t005u
    union all select 'erp_zserie', max(_charge_le) from raw.erp_zserie
    union all select 'garanties_clients', max(_charge_le) from raw.garanties_clients
    union all select 'garanties_enregistrements', max(_charge_le) from raw.garanties_enregistrements
    union all select 'garanties_reclamations', max(_charge_le) from raw.garanties_reclamations
    union all select 'telemetrie_sorties', max(_charge_le) from raw.telemetrie_sorties
)
```


© 2026 iAlexMG. Tous droits réservés.

```sql
select
    a.source,
    round(date_diff('minute', a.dernier, now()) / 60, 1) as heures,
    case
        when date_diff('minute', a.dernier, now()) > s.echouer_h * 60 then 'ERROR STALE'
        when date_diff('minute', a.dernier, now()) > s.avertir_h * 60 then 'WARN'
        else 'PASS'
    end as verdict_prevu
from ages as a
inner join seuils as s on s.source = a.source
order by a.source;
```

Mesuré le jour du chargement : dix lignes, 5,0 heures chacune, dix `PASS`, ce que dbt rend. Sur la copie vieillie où seules les réclamations ont 38 jours, la requête prédit 9 `PASS` et 1 `WARN`, 917 heures pour les réclamations, et dbt rend la même chose. Ce qu'elle ne sait pas faire : lire les seuils là où ils vivent. Elle les recopie, et le jour où le YAML change, elle ment ; dbt les lit dans le projet, qui est la seule source. *Théorie : `Theorie_08_AE.md`, section 8.*

P3. Un contrat sur le résumé. Le type réel d'abord, dans DuckDB :

```sql
select column_name, data_type
from information_schema.columns
where table_schema = 'presentation' and table_name = 'rpt_ventes_par_palier'
order by ordinal_position;
```

Sept colonnes : `annee` et `mois` en `BIGINT`, `nom_mois` et `palier` en `VARCHAR`, `lignes` en `BIGINT`, `valeur_nette` et `marge` en `DECIMAL(38,2)`. Le contrat, dans `_presentation.yml` :

```yaml
    config:
      contract:
        enforced: true
    columns:
      - name: annee
        data_type: bigint
      - name: mois
        data_type: bigint
      - name: nom_mois
        data_type: varchar
      - name: palier
        data_type: varchar
      - name: lignes
        data_type: bigint
      - name: valeur_nette
        data_type: decimal(38, 2)
        data_tests:
```


© 2026 iAlexMG. Tous droits réservés.

```
      - not_null
  - name: marge
    data_type: decimal(38, 2)
```

Le piège est sur les deux mesures. Dans le fait, elles sont en `decimal(18, 2)`, et on est tenté de recopier le type. Mais une somme élargit la précision : DuckDB rend un `DECIMAL(38,2)`. Avec `decimal(18, 2)` au contrat, la CI rend `PASS=127 ERROR=1 SKIP=2`, deux lignes de `data type mismatch` ; avec `decimal(38, 2)`, `PASS=129` sur la CI et `PASS=128` sur la production. Le type d'un contrat se lit dans la base, il ne se devine pas. *Théorie : `Theorie_08_AE.md`, section 9.*

P4. La CI mince, en local. L'état de référence, puis la sélection :

```
dbt parse
mkdir etat_prod
copy target\manifest.json etat_prod\
dbt ls --select state:modified+ --state etat_prod
```

Le dossier `etat_prod` joue ici l'état de production conservé ; Git le voit comme un dossier non suivi, et il se jette après l'essai.

Après avoir rendu la borne inclusive, la sélection rend 14 nœuds : `fct_ventes`, ses neuf tests de données, son test unitaire, le résumé, son test et l'`exposure`. C'est ce que la CI mince reconstruirait, au lieu des 130 nœuds du projet. Sur un entrepôt jetable neuf, `dbt build --select state:modified+ --state etat_prod --target ci --profiles-dir ci` rend `PASS=0 ERROR=1 SKIP=13`. Le verdict est juste, et pour la mauvaise raison : même avec la borne correcte, `dbt build --select fct_ventes` s'arrête sur le test unitaire, « Not able to get columns for unit test 'dim_concessionnaire' … because the relation doesn't exist ». Les parents non modifiés n'existent pas dans l'entrepôt neuf. Il manque de quoi les lire ailleurs, ce que fait `--defer` en allant les chercher dans l'environnement de production, et c'est pour cela que la CI mince suppose un entrepôt partagé et un état de production conservé. *Théorie : `Theorie_08_AE.md`, section 11.*

P5. La CI en une commande.

```
@echo off
rem verifier.cmd : la CI du depot, rejouee sur le poste en une commande, depuis la racine du depot.
if exist ci.duckdb del ci.duckdb
if exist ci.duckdb.wal del ci.duckdb.wal
dbt deps || exit /b 1
sqlfluff lint models tests snapshots || exit /b 1
python -c "import duckdb; duckdb.connect('ci.duckdb').execute(open('ci/structure_sources.sql', encoding='utf-8').read())" || exit
/b 1
dbt build --target ci --profiles-dir ci || exit /b 1
echo CI locale : au vert
```

`|| exit /b 1` arrête le script au premier code non nul, comme une étape de workflow en échec arrête la tâche. Mesuré : 27 secondes et `CI locale : au vert` sur `main`. Sur la branche de la borne, le linteur passe, `dbt build` rend `PASS=116 ERROR=1 SKIP=13`, et le script s'arrête avec le code 1, sans afficher la dernière ligne. Il ne remplace pas la CI : il ne tourne que si vous y pensez, ce qui est exactement le défaut que la CI corrige. *Théorie : `Theorie_08_AE.md`, section 3.*

© 2026 iAlexMG. Tous droits réservés.

P6. Un renommage sous contrat. La CI rend `PASS=117 ERROR=1 SKIP=12`, et le contrat dit les deux côtés :

```
| column_name   | definition_type | contract_type | mismatch_reason       |
| ------------- | --------------- | ------------- | --------------------- |
| cout          |                 | DECIMAL(18,2) | missing in definition |
| cout_standard | DECIMAL(18,2)   |               | missing in contract   |
```

La procédure : le renommage est une rupture d'interface, et il se traite comme telle. On l'annonce à ceux que l'`exposure` nomme ; on modifie le modèle et le contrat dans la même demande de tirage, pour que la revue voie la rupture écrite ; on la fusionne au moment convenu avec les consommateurs. Si personne ne peut suivre ce jour-là, on garde un temps les deux colonnes, l'ancienne et la neuve, et on retire l'ancienne plus tard. Retirer le contrat pour que la CI passe n'est pas une option, et le diagnostic E5 dit pourquoi. *Théorie : `Theorie_08_AE.md`, section 9.*

P7. Ce que le manifeste ne dit pas.

```python
import json

m = json.load(open("target/manifest.json", encoding="utf-8"))
modeles = [n for n in m["nodes"].values() if n["resource_type"] == "model"]
sans = sorted(n["name"] for n in modeles if not n["description"].strip())
print(len(modeles), "modeles,", len(sans), "sans description :", sans)
colonnes = [c for n in modeles for c in n["columns"].values()]
print(len(colonnes), "colonnes declarees,",
      sum(1 for c in colonnes if c["description"].strip()), "avec description")
sources = list(m["sources"].values())
print(len(sources), "sources,", sum(1 for s in sources if not s["description"].strip()), "sans description")
```

Sur le projet, après `dbt parse` : 31 modèles, dont 16 sans description, les douze vues `stg_*`, le contrôle du Cours 03 et les trois modèles intermédiaires ; 55 colonnes déclarées, dont 3 seulement avec une description ; 14 sources, dont 4 sans description. Un agent de documentation proposerait ces descriptions à partir du SQL et des noms, et c'est un bon usage. Ce qui reste humain : valider chacune, parce qu'une description inventée ne se distingue pas d'une vraie, et qu'elle entre ensuite dans ce que le prochain agent lira. *Théorie : `Theorie_08_AE.md`, section 10.*

P8. Le seuil qui se mesure.

```sql
with j as (
    select distinct (recu_le::timestamptz at time zone 'America/Toronto')::date as jour
    from raw.telemetrie_sorties
),

e as (
    select jour, lag(jour) over (order by jour) as avant
    from j
)
```


© 2026 iAlexMG. Tous droits réservés.

```
select count(*) as jours, max(jour - avant) as ecart_max_jours,
       arg_max(avant, jour - avant) as depuis, arg_max(jour, jour - avant) as jusqua
from e;
```

933 jours de réception, du 2023-09-09 au 2026-03-31, et un seul trou : rien entre le 10 et le 13 septembre 2023, trois jours d'écart. Le seuil de 72 heures aurait sonné, à la limite. Il ne faut pas pour autant l'élargir. Ces jours-là, le parc connecté publiait un ou deux messages par jour : aucune sortie les 11 et 12, donc aucun fichier, et un silence normal. Avec 7 644 véhicules connectés au 31 mars 2026, un jour sans message n'est plus un silence, c'est une panne. Un seuil se mesure sur la cadence de la source telle qu'elle est aujourd'hui, et se revoit quand elle change ; d'où la raison écrite à côté du chiffre. Côté langage, `arg_max(avant, jour - avant)` rend la valeur d'`avant` sur la ligne où l'écart est le plus grand : le Cours 09 de SQL l'enseigne sous son autre nom, `MAX_BY`, à la section 5. *Théorie : `Theorie_08_AE.md`, section 8.*

Diagnostics

E1. Au vert sur mon poste. Rangées sous `[sqlfluff]`, les deux clés sont ignorées sans un mot, et le templater cherche un profil là où dbt le cherche par défaut. Sur son poste, il trouve `~/.dbt/profiles.yml`, et compile avec la cible `dev` de ce profil personnel : le linteur passe, sur la mauvaise base. Sur le runner, qui n'a pas ce fichier : « Could not find profile named 'boreal_ae' ». Le profil de la CI, dans le dépôt, n'a été lu ni ici ni là. La parade est de ranger les deux clés sous `[sqlfluff:templater:dbt]`, et la leçon, de répéter la CI sur une copie neuve avant de conclure qu'elle marche. *Théorie : `Theorie_08_AE.md`, section 4.*

E2. La CI qui laisse passer dix-neuf millions. Elle passe parce qu'elle construit sur des tables vides : `PASS=129`, comme avant. Aucun test de données n'a de ligne à regarder, et le test unitaire ne porte que sur la jointure temporelle. Sur les vraies données, le fait passe à 82 842 lignes et 485 455 755,73 $, soit les 2 224 postes refusés et 19 531 142,59 $ de trop, et la réconciliation tombe sur 2 136 commandes. C'est elle qui l'attrape, là où tournent les données : sur l'entrepôt de production, dans la tâche mensuelle, ou sur la cible `dev` si le collègue l'avait lancée avant de proposer. Et même là, dbt construit un modèle avant de le tester : mesuré sur `dev`, quand la réconciliation tombe, les 82 842 lignes sont déjà écrites. La tâche échoue et l'alerte part, mais le fait fautif reste en place jusqu'à la correction. La CI de ce cours vérifie le code, pas les données, et ce défaut-là est un défaut de règle de métier. *Théorie : `Theorie_08_AE.md`, section 5.*

E3. L'alerte qui hurle. À 36 et 72 heures partout, la copie vieillie rend 10 `ERROR STALE` et le code 1, dont huit sur des sources parfaitement à jour. Passées en avertissements, les mêmes rendent le code 0, chaque jour : la tâche est verte, le journal est jaune, et personne ne le lit. L'équipe a perdu les deux choses que la fraîcheur donnait. Le signal, noyé dans huit fausses alertes sur dix. Et l'arrêt : quand la télémétrie cessera vraiment d'arriver, rien ne bloquera la publication. Le réglage juste, 35 et 40 jours sur les sources mensuelles, aurait donné sur la même copie 8 `PASS`, 1 `WARN` et 1 `ERROR STALE`, les deux vrais problèmes et eux seuls. *Théorie : `Theorie_08_AE.md`, section 8.*

E4. La fraîcheur dans la CI. L'étape rend 10 `ERROR STALE` et le code 1, à chaque exécution : sur l'entrepôt jetable, les tables sont vides, donc aucune n'a de chargement récent. La CI devient rouge en permanence, et une CI toujours rouge ne dit plus rien. La commande mesure les données, pas le code ; elle tourne là où les données arrivent, dans la tâche planifiée du Cours 07, juste avant `dbt build --target prod`, où une source périmée doit arrêter la chaîne. *Théorie : `Theorie_08_AE.md`, section 8.*

E5. Le contrat qui gênait. Sans le contrat, la CI rend `PASS=129`, et `fct_ventes` sort avec une colonne `cout_standard` et plus de `cout`. Rien dans le projet ne lit `cout` en aval : le résumé ne porte que la valeur nette et la marge, donc aucun modèle ne casse et aucun test ne bronche. Ce qui casse est dehors, chez Business

© 2026 iAlexMG. Tous droits réservés.

Intelligence, dont le tableau de bord lit le fait et le découvre le lendemain. Le contrat n'était pas un obstacle à contourner, c'était l'avertissement que la rupture allait sortir du projet ; le corrigé de la question P6 dit comment la mener. *Théorie : `Theorie_08_AE.md`, section 9.*

Questions à développer

A. Une CI sans données. La structure vide vérifie tout le code, et rien des données : la compilation, les références, les types, le contrat, les bornes que le test unitaire éprouve, et aucun des 98 tests de données, qui passent sur des tables vides. Elle ne coûte presque rien, ne fait sortir aucune donnée de l'entreprise, et ne prétend rien. Un échantillon versionné rendrait les tests de données utiles, au prix de trois dettes : quelqu'un doit le choisir et le tenir à jour, il fait entrer des données dans un dépôt, et un test qui passe sur l'échantillon ne dit rien de la production. La réponse n'est pas de choisir, c'est de faire tourner chaque vérification là où elle a un sens. La CI garde le code, sur la structure vide ; les tests de données et la fraîcheur tournent sur l'entrepôt de production, dans la tâche planifiée, et une erreur y arrête la publication. Le défaut du diagnostic E2 passe la CI et tombe à la prochaine exécution de production, sur la réconciliation : la tâche échoue et l'alerte part, même si le fait fautif, construit avant d'être testé, est déjà écrit. Sur une plateforme partagée, où la CI peut lire la vraie `RAW` en lecture seule, la frontière se déplace, et se paie. *Théorie : `Theorie_08_AE.md`, sections 5 et 11.*

B. Avertir ou échouer. Une erreur arrête quelque chose : une fusion, une tâche, une publication. Un avertissement s'adresse à un humain, et ne vaut que si un humain le lit. D'où la décision : échoue ce qui rend faux ce qu'on publierait, un grain qui se dédouble, une réconciliation qui ne tombe pas, une source périmée au-delà de sa cadence, un contrat rompu. Avertit ce qu'on veut voir sans arrêter la chaîne, un seuil connu qui bouge, les 23 articles sans année-modèle du Cours 05, une source en retard d'un jour. Les avertissements ont un lecteur nommé, et un moment : le matin, dans le journal de la tâche. Un projet où plus personne ne les lit est un projet où ils ne servent plus à rien, et c'est pire que de ne pas en avoir, parce qu'ils donnent l'impression qu'on surveille. Le signe se mesure : le jour où l'on passe les erreurs en avertissements pour faire taire le rouge, comme l'équipe du diagnostic E3, les seuils étaient faux, et c'est eux qu'il fallait revoir. *Théorie : `Theorie_08_AE.md`, section 8.*

C. Un agent qui relit les demandes de tirage. Ce qu'il peut attraper tient au manifeste et au `git diff` : un modèle sans description, un nom de table écrit en dur au lieu de `ref()`, un modèle neuf sans test, un changement à `fct_ventes` qui touche les douze nœuds qui en dépendent, une colonne retirée d'un modèle que l'`exposure` déclare. Ce qu'il ne peut pas attraper, ce sont les défauts de ce cours qui ne se voient ni dans le texte ni dans le graphe : une borne de `<` à `<=` qui a l'air prudente, un filtre retiré avec une bonne raison dans le message, un seuil de fraîcheur rond. Ceux-là, seuls un test unitaire, une réconciliation sur les données ou une mesure de la cadence les voient. Pour juger s'il vaut sa place, on le mesure comme un test : on lui soumet des demandes de tirage dont on connaît les défauts, les cinq diagnostics de ce cours par exemple, on compte ce qu'il trouve, ce qu'il rate et ce qu'il signale à tort, et on le garde s'il fait gagner du temps à la revue sans lui donner une fausse tranquillité. La formation AI Engineering fait de cette évaluation son Cours 05. *Théorie : `Theorie_08_AE.md`, sections 1 et 10.*

Mises en situation

M1. Transport scolaire de Rimouski. En premier, ce qui change le plus pour le moins cher : la branche `main` fermée aux poussées directes, tout changement passant par une demande de tirage, et une CI qui lance la forme et le fond à chacune, sur la structure vide des sources. Les tests existent déjà ; la CI ne fait que les lancer, et c'est tout ce qui manquait. Dans la même semaine, la tâche de production qui lance `dbt build` et alerte au premier test en échec, parce que les

© 2026 iAlexMG. Tous droits réservés.

deux rapports faux venaient de données que la CI n'aurait pas vues. Pour plus tard : le linteur en bloquant, qu'on introduit après un `fix` relu en entier, les contrats sur les modèles que les rapports lisent, et la CI mince. Ce que je refuse de sacrifier : la revue par un autre analyste. La CI vérifie ce qui se vérifie ; la borne qui a l'air juste, le filtre retiré pour une bonne raison, c'est un collègue qui les voit. *Théorie : `Theorie_08_AE.md`, sections 1, 2 et 5.*

M2. Fromagerie de Sherbrooke. L'authentification d'abord, et elle n'est pas facultative : la documentation de dbt, consultée le 2026-09-23, annonce la fin du mot de passe sur Snowflake au 31 août 2026. La CI passe à un utilisateur de service authentifié par une paire de clés, la clé privée dans un secret du dépôt, un rôle qui n'écrit que dans une base réservée à la CI et ne lit la `RAW` qu'en lecture seule. Ensuite, ce qu'elle reconstruit : la CI mince, `state:modified+`, avec le `manifest.json` de la production conservé à chaque exécution de production, et `--defer` pour lire dans la production les modèles qu'elle ne reconstruit pas. Elle exige un état de production publié et à jour, sans quoi la sélection se trompe. Enfin, ce qu'elle lit : sur une demande de tirage, la vraie `RAW` est un luxe qui se paie ; les tests de données complets tournent en production, et la CI peut se limiter aux modèles modifiés sur un échantillon de dates. Chaque économie se mesure sur la facture du mois suivant, pas sur une intuition. *Théorie : `Theorie_08_AE.md`, section 11.*

M3. Meunerie de Montmagny. La colonne est l'heure de chargement que l'entrepôt pose sur chaque ligne, pas la date du fichier, qui dit quand le fournisseur l'a daté et non quand il est arrivé. La cadence normale va de sept à huit jours, le mardi suivant un lundi férié : l'avertissement à 8 jours et demi, l'erreur à 10, pour laisser passer un mardi et attraper une semaine manquée. La vérification tourne dans la tâche planifiée, chaque jour, avant la construction qui utilise les prix, et une erreur arrête la publication des coûts de revient plutôt que de les calculer sur les prix de la semaine d'avant. Elle ne va pas dans la CI, qui n'a pas les données. À côté des chiffres, dans le YAML : « fichier du lundi, parfois du mardi après un férié ; avertir si le mardi est passé, échouer si une semaine manque ». Le jour où le fournisseur passera au quotidien, celui qui lira ces deux lignes saura que les seuils sont à revoir. *Théorie : `Theorie_08_AE.md`, section 8.*

© 2026 iAlexMG. Tous droits réservés.